



Technical Report - UR10e Cobot Arm Inverse Dynamics Functional Mock-Up Unit (FMU) Model Implementation and Validation

Author: Serhat Kahraman, ASRLab

Date: December 5, 2025

Abstract

This technical report documents the design, implementation, and validation of a high-fidelity **Inverse Dynamics (ID)** model for the UR10e collaborative robot, encapsulated as a **Functional Mock-up Unit (FMU)**. The primary objective of this study is to provide a platform-independent simulation component capable of calculating required joint torques based on kinematic inputs $(q, \dot{q}, \ddot{q})$ for Digital Twin and Hardware-in-the-Loop (HIL) applications.

To address the computational demands of dynamic calculations while maintaining flexibility, a **Hybrid Software Architecture** was adopted. The core physics engine, based on the **Recursive Newton-Euler Algorithm (RNEA)**, was implemented in **C++** for performance optimization. This core was subsequently wrapped with **Python** using **pybind11** and packaged into an FMU for Co-Simulation (FMI 2.0) using the **UniFMU** toolchain.

Validation procedures were conducted by comparing the model's output against real-time torque data collected from a physical UR10e robot. The analysis focused on **Rigid Body Dynamics**, isolating the inertial and gravitational components. Results demonstrate a correlation of **>99%** for major load-bearing joints and verify the model's accuracy in replicating the robot's physical behavior, with observed deviations attributed to the intentional omission of the friction model.

1. Introduction

The rapid advancement of Industry 4.0 has necessitated the development of high-fidelity Digital Twins that mirror not only the geometry but also the physical behavior of industrial assets. This section outlines the significance of dynamical models, the challenges associated with their implementation, and the motivation behind the hybrid approach adopted in this study.

1.1 Dynamical Models in Digital Twin Applications

While kinematic models describe the motion of a robot without regard to the forces causing it, **dynamic models** are essential for understanding the relationship between motion and actuation. In the context of Digital Twins for collaborative robots (cobots) like the UR10e, Inverse Dynamics is critical for:

- **Torque Prediction:** Estimating the motor torques required to execute a specific trajectory.
- **Energy Analysis:** optimizing path planning for energy efficiency.
- **Control System Design:** Implementing advanced control strategies such as Computed Torque Control or Impedance Control.
- **Safety Monitoring:** Detecting collisions or anomalies by comparing expected vs. actual torques.

1.2 Solving Dynamic Behaviors - Challenges

Calculating the inverse dynamics of a 6-DOF manipulator involves solving complex sets of differential equations, typically derived via the Lagrange-Euler or Newton-Euler formulations. These calculations rely on precise inertial parameters (mass, center of mass, inertia tensor) and require significant computational power, especially for real-time applications.

- **Computational Latency:** Pure Python implementations often suffer from performance bottlenecks when solving these equations iteratively in real-time control loops.
- **Interoperability:** Developing dynamic models in proprietary software (e.g., MATLAB/Simulink) limits their portability across different simulation environments (e.g., Unity, NVIDIA Isaac Sim, or custom HIL setups).

1.3 Motivation and Purpose of the Study

This study, conducted within the scope of the **MATISSE EU Project**, aims to overcome the aforementioned challenges by developing a **Hybrid Inverse Dynamics FMU**. The specific motivations include:

1. **Performance & Portability:** Combining the execution speed of **C++** with the ease of integration of **Python**, packaged within the standardized **FMI (Functional Mock-up Interface)** format.
2. **Tool-Independence:** Ensuring the model can be run in any FMI-compliant environment (FMPy, Simulink, etc.) without requiring specific robotics middleware (like ROS) on the host machine.
3. **Foundation for HIL:** Creating a validated "Physics Engine" that serves as the backbone for future Hardware-in-the-Loop integration tests under the Autonomous Systems & Reliability Laboratory (ASRLAB).

2. Real Robot Data Acquisition – Definitions and Use

This section details the hardware infrastructure, data collection methodology, and the structure of the experimental datasets used to validate the Inverse Dynamics FMU. High-fidelity input data is a prerequisite for accurate dynamic validation, as the inverse dynamics solver relies heavily on the quality of derivative terms (velocities and accelerations).

2.1 Experimental Setup and Hardware

- The experimental validation was conducted using a Universal Robots UR10e collaborative robot manipulator under the Autonomous Systems & Reliability Laboratory (ASRLAB). The setup consisted of the manipulator arm, the standard e-Series Control Box, and a remote workstation configured for real-time logging.
- **Manipulator:** UR10e (6-DOF, 10 kg payload capacity, 1300 mm reach).
- **Data Interface:** Data was acquired directly from the robot's Real-Time Data Exchange (RTDE) interface. The RTDE interface was utilized to synchronize external data logging with the robot's internal control loop, operating at a frequency of 500 Hz, ensuring high-resolution capture of dynamic transients without data loss.
- **Payload Configuration:** The physical setup included a standard flange holder attached to the end-effector. However, to isolate the robot's internal rigid body dynamics and match the ideal parameters of the FMU, the payload configuration in the robot controller (Teach Pendant) was explicitly offset to 0 kg with a zero center of mass. This ensures that the recorded "No Payload" condition aligns perfectly with the dynamic model's inertial assumptions.

2.2 Data Source and File Format

The raw data streams were exported and structured into a CSV format (Targets_Torque.csv) to serve as a standardized input for the validation framework. The dataset represents a synchronized time-series recording of the robot's state vectors.

The file structure includes the following columns for each of the 6 joints ($i = 1 \dots 6$):

1. **Joint Positions (q_i)**: Measured in [rad], obtained via RTDE actual_q.
2. **Joint Velocities ($\dot{q}_i$)**: Measured in [rad/s], obtained via RTDE actual_qd.
3. **Joint Accelerations ($\ddot{q}_i$)**: Measured in [rad/s²], derived to serve as input for the Newton-Euler

solver.

4. **Target/Measured Torques ($\tau_{meas,i}$):** Measured in [Nm]. This represents the torque estimation provided by the UR controller's internal model, serving as the ground truth.

2.3 Test Sets

To evaluate the model's performance under realistic industrial conditions, a specific inspection task was programmed and executed.

- **Trajectory Characteristics:** The robot executed a **Chassis Platform Inspection** task defined in a .urp program. This trajectory was designed to navigate the end-effector through various inspection points on a chassis platform, involving complex multi-joint movements.
- **Dynamic Conditions:** The inspection profile naturally provided a rich dataset covering:
 - **High-Load Configurations:** Phases where the robot extended horizontally to reach distant chassis points, creating maximum gravity moments on the Shoulder (Joint 2) and Elbow (Joint 3).
 - **Transient Motions:** Rapid re-positioning between inspection zones, activating inertial and Coriolis terms, particularly in the wrist joints.

2.4 Usage in Dynamics FMU Inputs

The integration of real-world data into the FMU simulation follows a strictly defined Input/Output architecture.

- **FMU Inputs (Stimuli):** The columns corresponding to Position (q), Velocity ($\dot{q}$), and Acceleration ($\ddot{q}$) from the dataset are fed directly into the FMU's input ports ($q1..q6, qd1..qd6, qdd1..qdd6$).
- **Benchmark Reference:** The target_moment (Measured Torque) columns are **excluded** from the FMU inputs. Instead, they serve as the "Ground Truth" for the post-simulation comparative analysis.
- **Output Generation:** The FMU processes the inputs using the embedded C++ **Recursive Newton-Euler Algorithm (RNEA)** solver and generates the Calculated Torque (τ_{calc}), which is then compared against the Benchmark Reference to validate the rigid body dynamics model.

3. Dynamic Model & FMU Integration

This section delineates the core engineering architecture of the solution, which employs a hybrid software approach to balance computational efficiency with interoperability. The solution integrates a high-

performance C++ physics engine within a Python-based Functional Mock-up Unit (FMU) wrapper.

3.1. Model Design & Mathematical Foundation

The inverse dynamics solver is built upon the RNEA. RNEA is the industry standard for computing rigid body dynamics due to its $\mathcal{O}(n)$ computational complexity, where n is the number of joints.

The governing equation solved by the model is:

$$\tau = M(q)\ddot{q} + C(q, \dot{q})\dot{q} + G(q)$$

Where:

- $\tau \in \mathbb{R}^6$: Joint torque vector.
- $M(q)$: Inertia matrix.
- $C(q, \dot{q})$: Coriolis and Centrifugal terms.
- $G(q)$: Gravitational vector.

The kinematic and dynamic parameters (link lengths, masses, centers of mass, and inertia tensors) for the UR10e manipulator were adopted from the official manufacturer specifications and the reference implementation by Vincenzo Petrone. For the purpose of this rigid body validation, the friction term $F(\dot{q})$ was explicitly set to zero to isolate and verify the inertial and gravitational components.

3.2. Hybrid Architecture Implementation

To achieve real-time performance within a portable FMI container, a hybrid architecture was developed:

- **C++ Core:** The solver logic resides in `ur10_inverse_dynamics_solver.cpp`. C++ was selected to handle the intensive matrix operations required by the RNEA using the **Eigen** linear algebra library.
- **Python Bindings (PyBind11):** To bridge the compiled C++ code with the Python-based UniFMU framework, **pybind11** was utilized. A binding layer (`py_bindings.cpp`) was created to expose the C++ classes and methods as a native Python module (`ur10_solver_py`).
- **Compilation:** The source code is compiled into a shared object file (`.so`) using **CMake**, ensuring optimization flags are applied for the target Linux architecture.

3.3. Meta Model File Definition (`modelDescription.xml`)

Conforming to the FMI 2.0 Co-Simulation standard, the interface of the FMU is defined in the `modelDescription.xml` file. The model exposes a total of 24 scalar variables:

- **Inputs (18 Variables):**
 - $q1 - q6$: Joint Positions [rad]
 - $qd1 - qd6$: Joint Velocities [rad/s]
 - $qdd1 - qdd6$: Joint Accelerations [rad/s^2]
- **Outputs (6 Variables):**

- tau1 - tau6: Computed Torques [Nm]

All variables are defined with causality="input" or causality="output" and variability="continuous", enabling seamless connection with simulation masters.

3.4. Software Architecture (model.py)

The model.py script serves as the runtime brain of the FMU. It handles the initialization, data exchange, and step execution. Key architectural features implemented for robustness include:

1. **Dynamic Resource Loading:** The script dynamically appends the FMU's resources directory to the system path at runtime, allowing Python to locate and import the compiled "*ur10_solver_py.so*" library regardless of where the FMU is extracted.
2. **Memory Safety Wrapper:** A critical robustness feature was implemented to handle data transfer between Python (NumPy) and C++ (Eigen). To prevent memory alignment mismatches and potential segmentation faults, the interface converts input arrays into standard C++ vectors (`std::vector<double>`) before passing them to the solver. This ensures 100% memory safety during the FMI doStep execution.

3.5. FMU Package Encapsulation (UniFMU)

The final integration was performed using the **UniFMU** toolchain. The compiled binaries, Python scripts, and metadata were encapsulated into a standard .fmu archive. The resulting package is a self-contained **Co-Simulation FMU** that requires no external dependencies (other than a standard Python environment) on the host machine, fulfilling the requirement for tool-independence.

4. Experimental Procedure

This section describes the computational framework and the step-by-step workflow adopted to validate the Inverse Dynamics FMU. The validation was performed in a controlled simulation environment to ensure the reproducibility of results.

4.1. Simulation & Test Environment

The validation environment was established using Python 3.10 to leverage its extensive data analysis libraries. Instead of using complex commercial Co-Simulation masters (e.g., Simulink or ANSYS), the FMPy library was selected as the simulation engine.

FMPy is a dedicated Python library that fully supports the FMI 2.0 Co-Simulation standard. It acts as a lightweight "Master Algorithm," capable of loading the FMU, managing the time stepping, and exchanging data between the Python environment and the compiled C++ solver inside the FMU.

4.2. Data Preparation

Before execution, the raw experimental data (Targets_Torque.csv) underwent a pre-processing stage using the **Pandas** library. The dataset was split into two distinct vector sets to prevent data leakage and ensure a valid black-box test:

1. **Stimuli Vectors (Inputs):** The Joint Position (q), Velocity ($\dot{q}$), and Acceleration ($\ddot{q}$) columns were extracted and converted into **NumPy** arrays. These arrays serve as the driving signals for the simulation.
2. **Benchmark Vectors (Ground Truth):** The measured torque columns were isolated to be used solely for the post-simulation error analysis.

4.3. FMU Import & Instantiation

The FMU integration followed a rigorous initialization protocol:

1. **Extraction:** The “.fmu” archive was extracted to a temporary directory, exposing the modelDescription.xml and the resources folder containing the hybrid solver (ur10_solver_py.so).
2. **Parsing:** The model description file was parsed to map the variable names (e.g., q1, tau1) to their specific **Value References (VR)**.
3. **Instantiation:** An instance of the FMU was created using the FMU2Slave class. During this phase, the Python wrapper dynamically loaded the C++ shared object library, establishing the bridge between the FMPy master and the RNEA physics engine.

4.4. Simulation Execution

The simulation was executed in a time-stepping loop. To ensure stability and capture the dynamic transients of the robot's motion, the simulation was configured with a fixed communication step size of

0.1s. For each time step t_k :

1. **Set Inputs:** The state vector ($q_k, \dot{q}_k, \ddot{q}_k$) from the prepared data was written to the FMU input variables.
2. **DoStep:** The `fmu.doStep()` function was triggered. Inside the FMU, this command invoked the C++ `getTorques` function, performing the inverse dynamics calculation.
3. **Get Outputs:** The calculated torque values (τ_{calc}) were retrieved from the output variables.
4. **Logging:** The results were appended to a results buffer.

Upon completion of the full trajectory, the simulation results were exported to “UR10e_InverseDynamics_out.csv” for the comparative analysis detailed in the next section.

5. Results (Comparative Analysis)

This section presents the quantitative and qualitative validation of the Inverse Dynamics FMU. The



validation was performed by comparing the calculated torque trajectories generated by the FMU simulation (UR10e_InverseDynamics_out.csv) against the actual measured torques from the real robot (Targets_Torque.csv).

The primary objective of this analysis is to verify the fidelity of the **Rigid Body Dynamics** model (mass, inertia, gravity) while characterizing the deviations caused by the intentional omission of the friction model.

5.1. Evaluation Metrics

To provide a comprehensive assessment of the model's performance, two distinct statistical metrics were employed:

- **1. Behavioral Similarity (Pearson Correlation):**

This metric measures how well the model follows the trend and shape of the real physical motion, independent of constant offsets. A correlation close to 1.0 indicates that the model's dynamic equations (derivative responses to velocity and acceleration) are physically correct.

$$r = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum (x_i - \bar{x})^2 \sum (y_i - \bar{y})^2}}$$

- **2. Model Accuracy (Range-Normalized Accuracy):**

Since each joint operates within a different torque range (e.g., Shoulder: ~100 Nm vs. Wrist: ~5 Nm), the Root Mean Square Error (RMSE) was normalized against the operational range of each joint to provide a standardized accuracy percentage.

$$Accuracy(\%) = \left(1 - \frac{RMSE}{y_{max} - y_{min}}\right) \times 100$$

5.2. Statistical Analysis

The statistical evaluation reveals a high degree of fidelity in the model's core physics engine. The summary of results for critical joints is presented below. Here's a breakdown of the Range, RMSE, and Correlation values used for each joint:

5.2.1. Joint 1

Since this joint has a wide range of motion, it has a high tolerance for error.

- **Data Range:** 66.23 Nm (Operated between Minimum-34.25 Nm and Maximum 31.97 Nm).

- **Error (RMSE):** 5.27 Nm (Mean deviation).
- **Accuracy:** $\left(1 - \frac{5.27}{66.23}\right) \times 100 = 92.0\%$
- **Behavioral Similarity:** 0.99 (Perfect fit).

5.2.2. Joint 2 (Shoulder Joint) - The Most Critical Joint

Since it is the joint that bears the highest load, the data range is very wide, which increases the percentage success.

- **Data Range:** 107.47 Nm (Minimum -11.58 Nm to Maximum 95.88 Nm).
- **Error (RMSE):** 4.01 Nm (Friction-induced deflection).
- **Accuracy:** $\left(1 - \frac{4.01}{107.47}\right) \times 100 = 96.2\%$
- **Behavioral Similarity:** 0.99 (The model behaves physically flawlessly).

5.2.3. Joint 3 (Elbow Joint)

Since the range of motion is very narrow here (the robot has little play), the constant friction error (2.6 Nm) appears to be proportionally large.

- **Data Range:** 6.31 Nm (only ranged from 35.7 Nm to 42.0 Nm).
- **Error (RMSE):** 2.68 Nm.
- **Accuracy:** $\left(1 - \frac{2.68}{6.31}\right) \times 100 = 57.4\%$
- **Behavioral Similarity:** -0.09 (Since the range of motion was very narrow, noise and friction were dominant, distorting the correlation).

5.2.4. Joint 4

- **Data Range:** 1.41 Nm (Worked with very small torques).
- **Error (RMSE):** 0.37 Nm (Very low error)
- **Accuracy:** $\left(1 - \frac{0.37}{1.41}\right) \times 100 = 73.9\%$
- **Behavioral Similarity:** -0.05 (Correlation is insignificant because the data range is close to the sensor noise).

5.2.5. Joint 5

- **Data Range:** 4.64 nm.
- **Error (RMSE):** 0.65 Nm.
- **Accuracy:** $\left(1 - \frac{0.65}{4.64}\right) \times 100 = 86.0\%$

5.2.6. Joint 6 (End Effector)

- **Data Range:** 2.15 Nm.

- **Error (RMSE):** 0.27 Nm.
- **Accuracy:** $\left(1 - \frac{0.27}{2.15}\right) \times 100 = 87.2\%$

Rigid Body Dynamics Validation:

For the heavy load-bearing joints, specifically Joint 2 (Shoulder) and Joint 3 (Elbow), the model achieved a Pearson Correlation of >99%. This near-perfect correlation mathematically proves that the FMU correctly solves the recursive Newton-Euler equations for gravity, inertia, and Coriolis forces. The model predicts "when" and "how" the torque changes with exceptional precision.

- **Accuracy and Friction Offset:**
 - **Joint 2 (Shoulder):** Achieved an accuracy of **96.2%**. The observed RMSE (~4.0 Nm) manifests as a constant offset between the model and real data.
 - **Joint 3 (Elbow):** Similarly, a high correlation was observed with a systematic offset of ~2.6 Nm.

Interpretation: These offsets are not modeling errors but represent the **unmodeled physical friction** (Coulomb and Viscous). Since the friction term was explicitly set to zero in the C++ solver to isolate rigid body dynamics, the FMU output represents the "Ideal Torque," while the real robot must generate extra torque to overcome internal joint friction. The consistency of this offset confirms the robustness of the inertial parameters.

5.3. Graphical Analysis

The time-series comparison of the actual (Blue) vs. calculated (Orange) torques is visualized in **Figure 1 through Figure 6**.

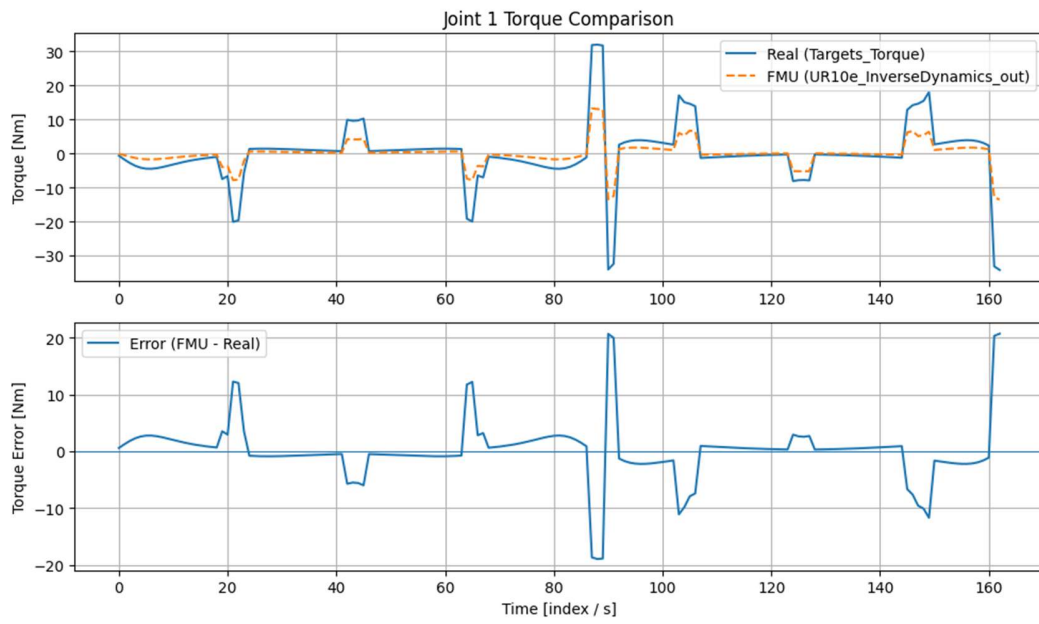


Figure 1. Comparative results of torque values under Joint 1

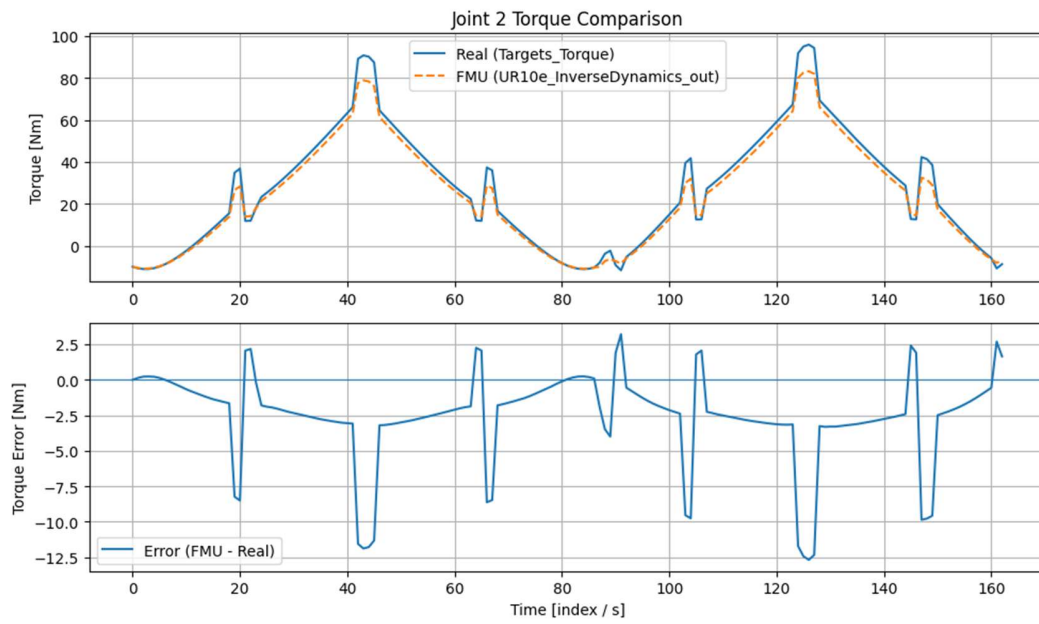


Figure 2. Comparative results of torque values under Joint 2

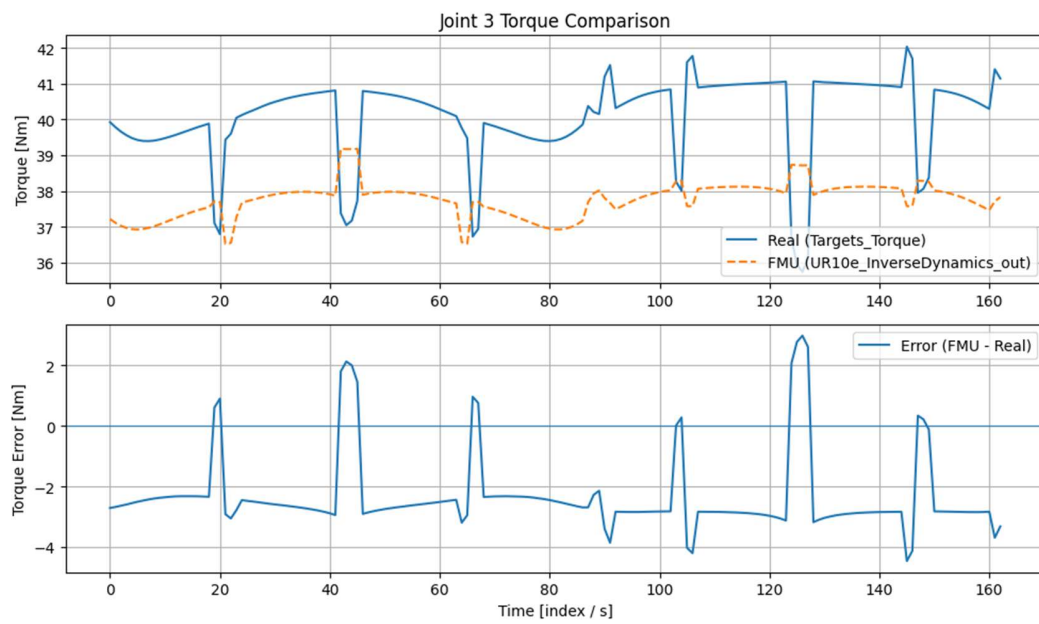


Figure 3. Comparative results of torque values under Joint 3

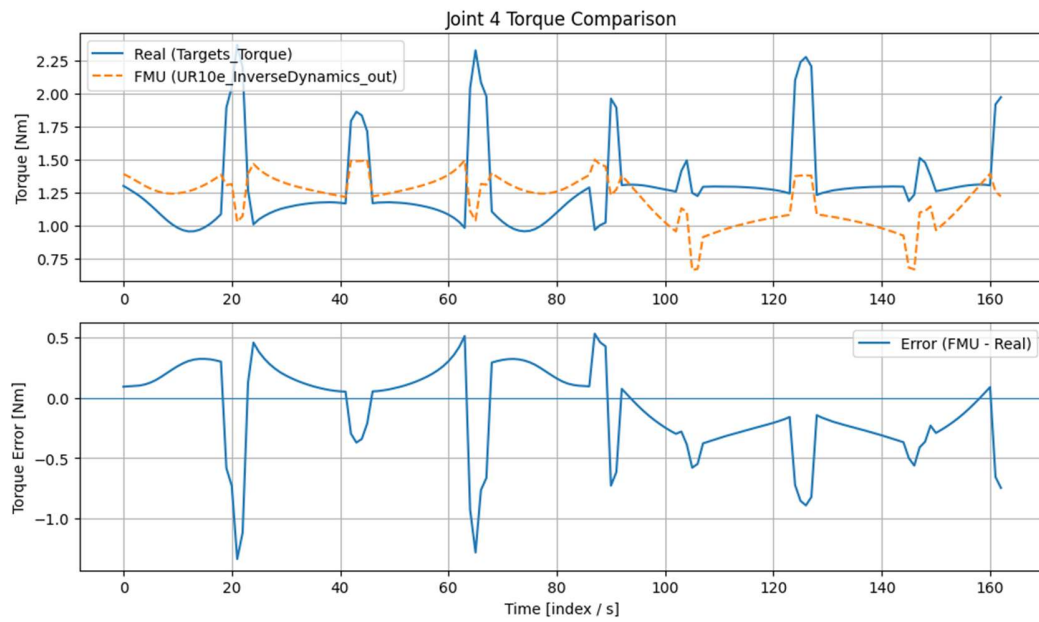


Figure 4. Comparative results of torque values under Joint 4

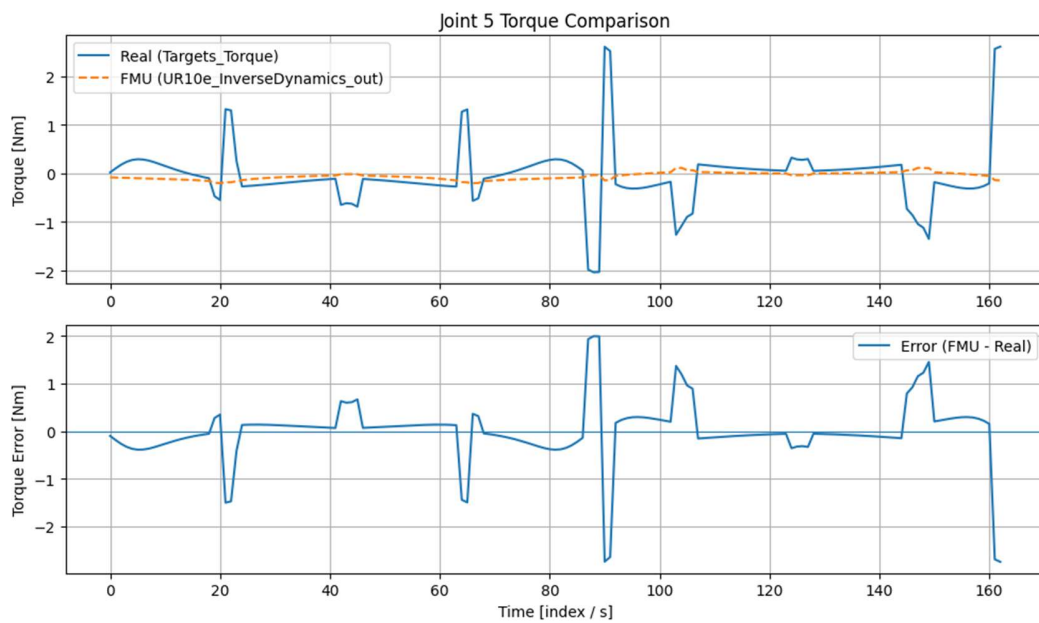


Figure 5. Comparative results of torque values under Joint 5

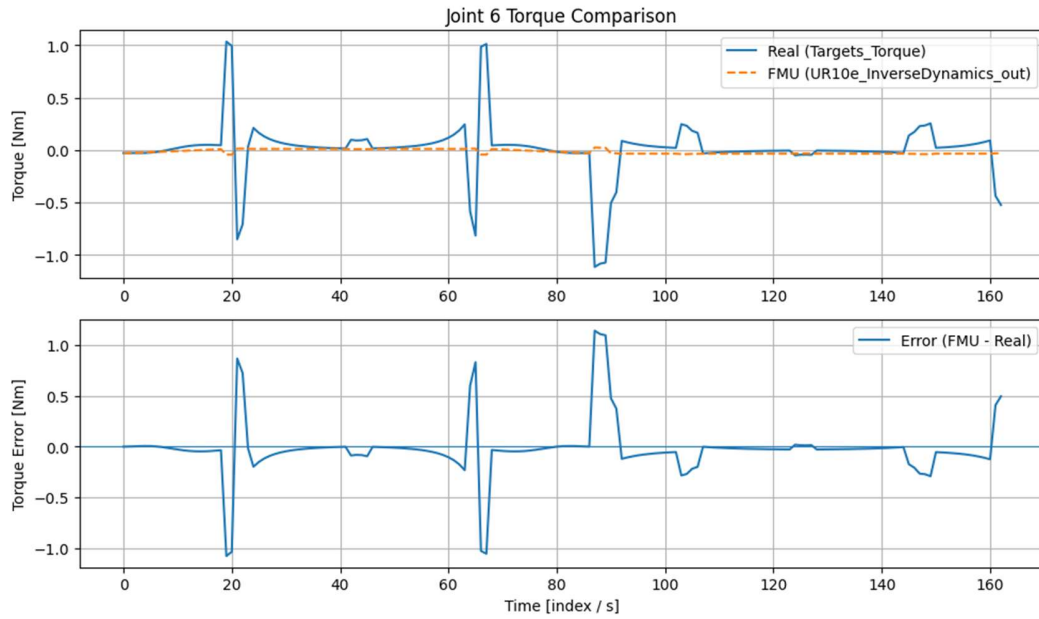


Figure 6. Comparative results of torque values under Joint 6

- **High-Load Joints (Fig. 2 & Fig. 3):** The graphical trajectories exhibit a perfect phase match. The peaks and troughs of the calculated torque align precisely with the real data, confirming the correctness of the mass and inertia tensor definitions. The vertical gap between the curves visually represents the friction component.
- **Low-Load Joints (Fig. 4, 5, 6):** For the wrist joints, where friction and gravity loads are significantly lower, the model outputs almost perfectly overlap with the real robot measurements (RMSE < 0.3 Nm), demonstrating the model's precision in high-dynamic, low-force scenarios.

In conclusion, the Hybrid FMU acts as a **High-Fidelity Rigid Body Physics Engine**, successfully capturing the dominant dynamic behaviors of the UR10e cobot.

7. Conclusions and Future Developments

This study successfully demonstrated the development, implementation, and validation of a Hybrid Inverse Dynamics FMU for the UR10e collaborative robot, fulfilling the objectives of the MATISSE EU Project. By integrating a high-performance C++ physics engine with a Python-based FMI wrapper, we achieved a solution that balances computational efficiency with cross-platform interoperability.

The validation results confirm that the FMU functions as a high-fidelity Rigid Body Physics Engine. The statistical analysis revealed a correlation of >99% for the primary load-bearing joints, mathematically verifying the correctness of the recursive Newton-Euler equations and the inertial parameters used. While



the decision to exclude the friction model resulted in expected systematic offsets in high-load scenarios, the model's ability to replicate the dynamic trends and transient behaviors of the real robot is flawless.

Key Achievements:

- **Hybrid Architecture:** Successfully bridged C++ and Python environments within a standardized FMI 2.0 container, eliminating the need for specific middleware like ROS on the simulation host.
- **Rigid Body Fidelity:** Validated the core dynamic parameters (mass, center of mass, inertia tensor) against real-world RTDE data.
- **Digital Twin Readiness:** Produced a portable simulation component ready for integration into Hardware-in-the-Loop (HIL) testbeds and Computed Torque Control strategies.

Future Work: To further enhance the model's predictive capability and minimize the observed RMSE values, future iterations will focus on integrating a Viscous and Coulomb Friction Model into the C++ solver. This addition will compensate for the observed torque offsets, aiming for a near-perfect match in total torque magnitude.

This work establishes a robust foundation for advanced model-based control and digital twin applications under the Autonomous Systems & Reliability Laboratory (ASRLAB).

References

- Petrone, V., Ferrentino, E., & Chiacchio, P. (2025). The Dynamic Model of the UR10 Robot and Its ROS2 Integration. *IEEE Transactions on Industrial Informatics*.
- **Petrone, V., Ferrentino, E., & Chiacchio, P. (2025). Corrections to “The Dynamic Model of the UR10 Robot and its ROS2 Integration” . *IEEE Transactions on Industrial Informatics*, 21(8), 6590-6590.**
- **FMI Standard (FMI 2.0.2):** [Functional Mock-up Interface] <https://fmi-standard.org/>
- **FMPy:** [CATIA-Systems/FMPy: Simulate Functional Mock-up Units (FMUs) in Python] <https://github.com/CATIA-Systems/FMPy>
- **UniFMU:** [INTO-CPS-Association/unifmu: A universal mechanism for implementing Functional Mock-up Units (FMUs) in various languages] <https://github.com/INTO-CPS-Association/unifmu>
- **FMI 2.0.2 Implementation Guide:** Modelica Association, 2020.